Robotic Arm Forward and Inverse Kinematics Solution

Robotic Arm Forward and Inverse Kinematics Solution

1. Course Content
2. Kinematics CLI Command Format
3. Case Demonstration
    3.1 View Robotic Arm Kinematics Information
    3.2 Robotic Arm Forward Kinematics Solution
    3.3 Robotic Arm Inverse Kinematics Solution
4. Core Source Code Analysis

1. Course Content

- This course implements the calculation of robotic arm forward kinematics and inverse kinematics through CLI commands.
- Forward kinematics calculates the pose of the end-effector from the angle value of each servo of the robotic arm
- Inverse kinematics, on the other hand, calculates the angle value of each servo from the end-effector pose. Both play a crucial role in 3D spatial grasping. The forward kinematics algorithm can tell the current pose of the robotic arm end-effector
- The inverse kinematics algorithm can calculate, for the robotic arm end-effector to reach a target pose, what the angle of each servo of the robotic arm should be. Before grasping, this algorithm needs to be called to calculate the servo values, and then the servos are controlled to move to the grasping pose.

2. Kinematics CLI Command Format

- Forward kinematics solution: joint1—joint5 are the values of robotic arm joints 1-5

```
kin_measure_cal fk joint1 joint2 joint3 joint4 joint5
```

- Inverse kinematics solution: --x --y --z --roll --pitch --yaw are the pose of the robotic arm end-effector TCP coordinate frame

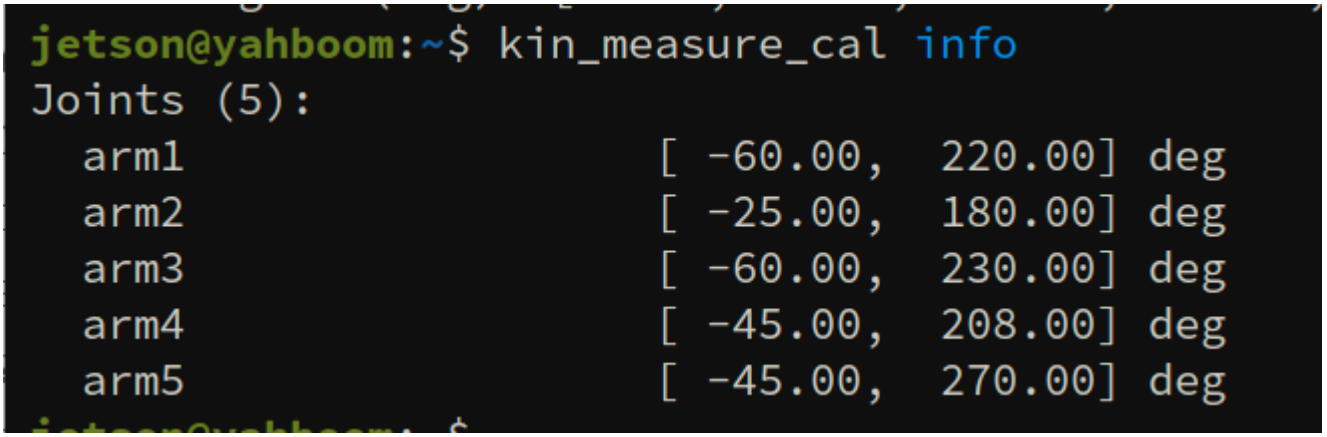
```
kin_measure_cal ik --x  --y  --z  --roll --pitch --yaw
```

3. Case Demonstration

3.1 View Robotic Arm Kinematics Information

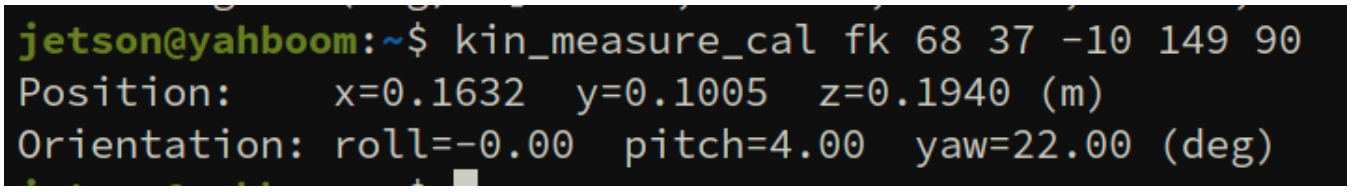
```
kin_measure_cal info
```

Here joint(5) is the number of joints that contribute degrees of freedom to the end-effector pose, i.e., joint 1-joint 5; after arm1-arm5 are the joint limits used when calculating kinematics (slightly larger than the actual joint limits, ensuring that the kinematics is solvable)



3.2 Robotic Arm Forward Kinematics Solution

```
kin_measure_cal fk 68 37 -10 149 90
```



3.3 Robotic Arm Inverse Kinematics Solution

> **Important**
>
> - The end-effector of this robotic arm configuration has only 5 degrees of freedom (lacking yaw), and the end-effector pose roll is determined only by joint 5 (servo No. 5), so the actual inverse solution is simplified to a 4-degree-of-freedom solution: only the end-effector position x, y, z and pitch need to be specified, while the weights of roll and yaw are ignored during the inverse solution
> - The robotic arm's low-level firmware can only use integers when receiving robotic arm angles, so when actually applying the inverse solution result (a floating-point number), it is rounded to an integer for use

- Perform the inverse solution using the pose from the forward solution result

```
kin_measure_cal ik --x 0.1632  --y 0.1005  --z 0.1940 --pitch 4.00
```

- You can see that the angles are basically consistent with the forward solution

```
jetson@yahboom:~$ kin_measure_cal ik --x 0.1632  --y 0.1005  --z 0.1940 --pitch 4.00
Joint angles (deg): [68.01, 37.05, -10.05, 148.99, 90.00]
```

4. Core Source Code Analysis

- Program source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/robot_util/cli/kin_measure_cal
```

- Robotic arm low-level forward and inverse solution library

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/robot_util/robot_util/arm_kinemarics.py
```

Global variables and helper functions:

- `DEFAULT_URDF`: default URDF model file path, pointing to `rosmaster_m1pro.urdf.xacro`, used to parse kinematic parameters such as the link lengths and joint limits of the robotic arm
- `_build_kin(args)`: creates an `ArmKinematics` kinematics solver instance, specifying the base coordinate frame as `measure` and the end-effector coordinate frame as `gripper_tcp`

cmd_fk function (forward kinematics):

- `kin.forward_kinematics(joints)`: takes 5 joint angle values (degrees) as input and calculates the pose of the end-effector TCP in the measure coordinate frame
- `result.success`: determines whether the solution succeeded; if it failed, outputs an error message and exits
- `pose.x / pose.y / pose.z`: end-effector position coordinates (unit: meters), relative to the measure coordinate frame
- `pose.roll / pose.pitch / pose.yaw`: end-effector pose Euler angles (unit: degrees), describing the spatial orientation of the gripper

cmd_ik function (inverse kinematics):

- `kin.inverse_kinematics(...)`: takes the target pose (x/y/z position and roll/pitch/yaw orientation) as input and solves for each joint angle
- `seed_angles_deg`: optional seed joint angles, providing an initial guess for the solver, which helps select the solution closest to the current pose among multiple solutions
- `allow_approximate=True`: allows approximate solutions; when the target pose is kinematically unreachable, returns the closest feasible solution
- `pitch_search_space=20.0`: pitch angle search space (degrees), searching for a feasible pitch value within this range

- `result.approximate`: marks whether the solution is approximate; if so, outputs the position error
- `result.adjusted_pitch`: if the solver automatically adjusted the pitch angle, outputs the values before and after adjustment
- `result.data`: the 5 joint angle values obtained from the solution (degrees)

cmd_info function (joint information):

- `kin.NUM_JOINTS`: number of robotic arm joints
- `kin._joint_names`: list of each joint name
- `kin._joint_limits_deg`: angle limit range of each joint (degrees), parsed from the URDF file

main function (CLI entry):

- `argparse.ArgumentParser`: creates the command-line argument parser
- `sub.add_subparsers`: adds a subcommand parser, supporting three subcommands: `fk` (forward solution), `ik` (inverse solution), and `info` (joint information)
- `fk` subcommand: the `joints` argument receives space-separated joint angle values (degrees)
- `ik` subcommand: `--x/--y/--z` are required target position arguments (meters), `--roll/--pitch/--yaw` are optional target orientation arguments (degrees), and `--seed` is an optional seed angle
- `{'fk': cmd_fk, 'ik': cmd_ik, 'info': cmd_info}[args.command](args)`: calls the corresponding handler function according to the subcommand name

```
DEFAULT_URDF
=os.path.join(get_package_share_directory("yahboomcar_description"),"urdf","rosmaster_m1pro.
urdf.xacro")


def _build_kin(args) -> ArmKinematics:
    return ArmKinematics(urdf_path=args.urdf, base_link="measure", end_link="gripper_tcp")

def cmd_fk(args):
    """Forward kinematics: joint angles -> end-effector pose."""
    kin = _build_kin(args)
    joints = [float(x) for x in args.joints]
    result = kin.forward_kinematics(joints)
    if result.success:
        pose = result.data
        print(f"Position:    x={pose.x:.4f}  y={pose.y:.4f}  z={pose.z:.4f} (m)")
        print(f"Orientation: roll={pose.roll:.2f}  pitch={pose.pitch:.2f}  yaw=
{pose.yaw:.2f} (deg)")
    else:
        print(f"FK failed: {result.message}", file=sys.stderr)
        sys.exit(1)


def cmd_ik(args):
    """Inverse kinematics: target pose -> joint angles."""
    kin = _build_kin(args)
    result = kin.inverse_kinematics(
```

```
        target_x=args.x,
        target_y=args.y,
        target_z=args.z,
        target_roll=args.roll,
        target_pitch=args.pitch,
        target_yaw=args.yaw,
        seed_angles_deg=[float(x) for x in args.seed] if args.seed else None,
        allow_approximate=True,
        pitch_search_space=20.0
    )
    if result.success:
        joints_str = ', '.join(f'{j:.2f}' for j in result.data)
        print(f"Joint angles (deg): [{joints_str}]")
        if result.approximate:
            print(f"NOTE: approximate solution, position error = {result.position_error:.4f}
m")
        if result.adjusted_pitch:
            print(f"NOTE: adjusted pitch angle from {args.pitch:.2f} to
{result.adjusted_pitch:.2f} deg")
    else:
        print(f"IK failed: {result.message}", file=sys.stderr)
        sys.exit(1)


def cmd_info(args):
    """Show joint names and limits from URDF."""
    kin = _build_kin(args)
    print(f"Joints ({kin.NUM_JOINTS}):")
    for name, (lo, hi) in zip(kin._joint_names, kin._joint_limits_deg):
        print(f"  {name:20s}  [{lo:7.2f}, {hi:7.2f}] deg")


def main():
    parser = argparse.ArgumentParser(
        description="Robot arm kinematics calculator")
    parser.add_argument(
        '--urdf', default=DEFAULT_URDF,
        help='Path to URDF/xacro file (default: rosmaster_m1pro)')

    sub = parser.add_subparsers(dest='command', required=True)

    # --- fk ---
    p_fk = sub.add_parser('fk', help='Forward kinematics: joints -> pose')
    p_fk.add_argument(
        'joints', nargs='+', metavar='ANGLE',
        help='Joint angles in degrees (space separated)')

    # --- ik ---
    p_ik = sub.add_parser('ik', help='Inverse kinematics: pose -> joints')
    p_ik.add_argument('--x', type=float, required=True, help='Target x (m)')
    p_ik.add_argument('--y', type=float, required=True, help='Target y (m)')
    p_ik.add_argument('--z', type=float, required=True, help='Target z (m)')
```

```
    p_ik.add_argument('--roll', type=float, default=0.0, help='Target roll (deg, default
0)')
    p_ik.add_argument('--pitch', type=float, default=0.0, help='Target pitch (deg, default
0)')
    p_ik.add_argument('--yaw', type=float, default=0.0, help='Target yaw (deg, default 0)')
    p_ik.add_argument('--seed', nargs='+', metavar='ANGLE',
                      help='Seed joint angles in degrees (optional)')

    # --- info ---
    sub.add_parser('info', help='Show joint names and limits')

    args = parser.parse_args()
    {'fk': cmd_fk, 'ik': cmd_ik, 'info': cmd_info}[args.command](args)
```